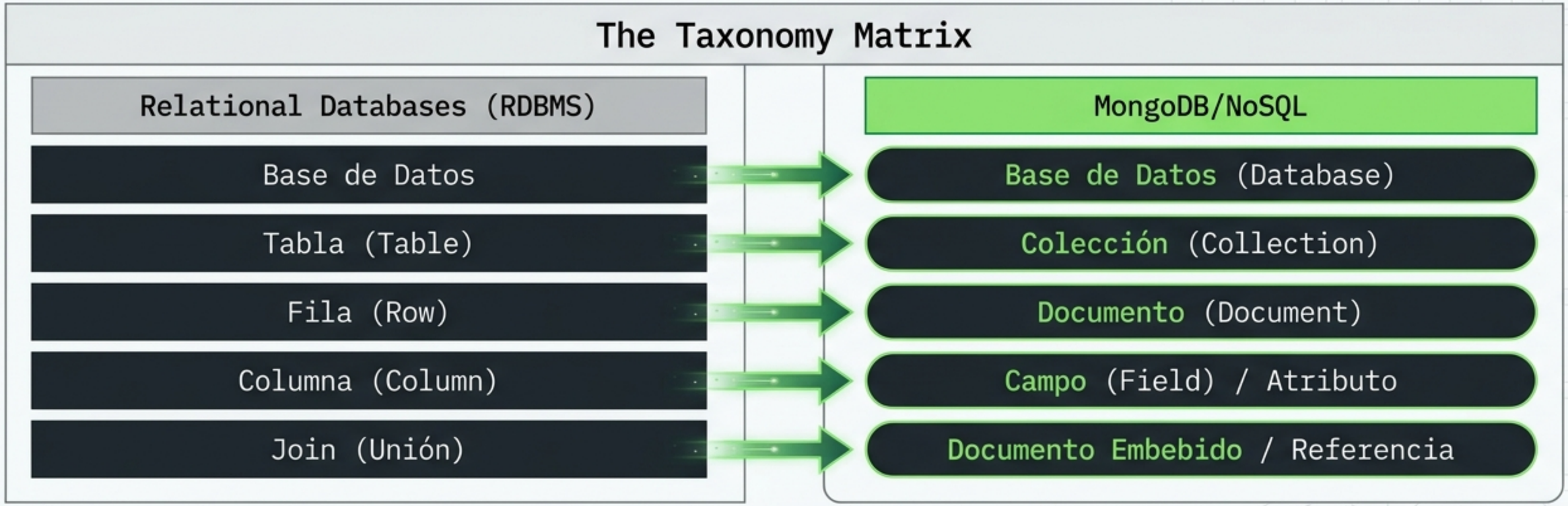
> mongo --Arquitectura de Datos Modernos

Guía de Referencia y Diseño NoSQL para Desarrolladores

Tecnólogo en Análisis y Desarrollo de Software SENA

Gemini Notebook

Traduciendo el Modelo Mental: SQL a NoSQL



Las **bases de datos documentales** utilizan una aproximación **Clave-Valor** **avanzada**, almacenando datos como representaciones semiestructuradas (JSON/BSON) sin un esquema rígido predefinido.

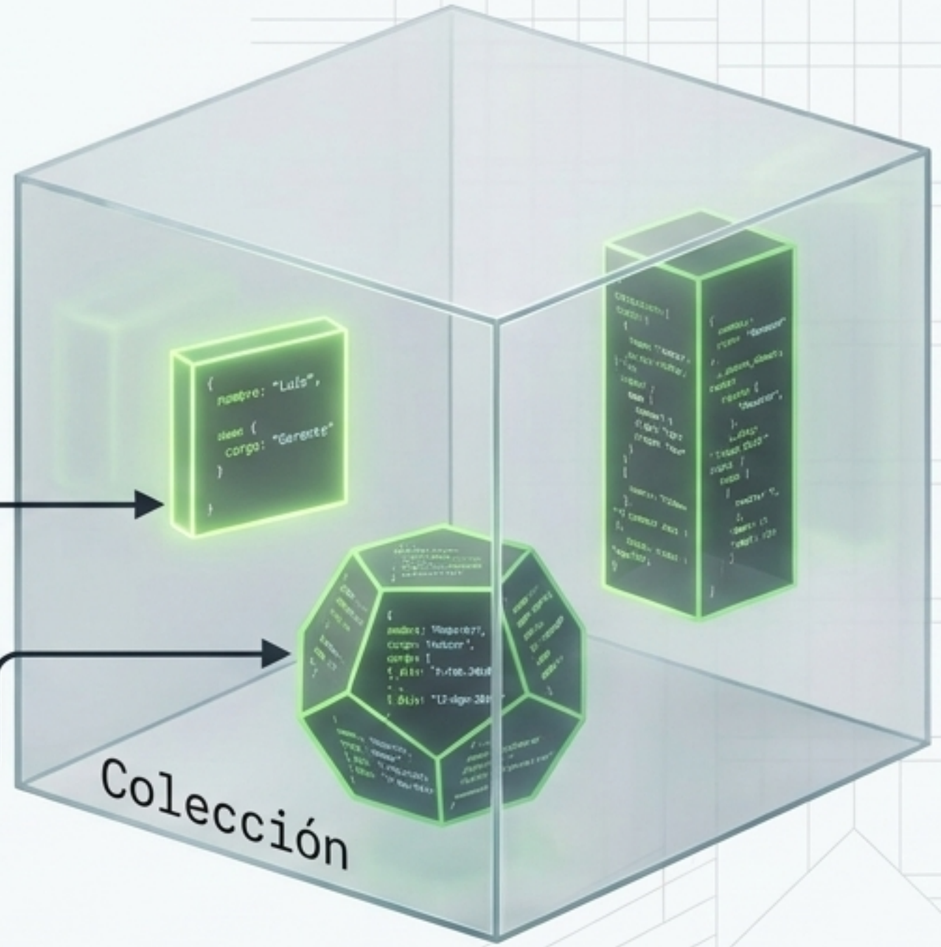
Gemini Notebook

Flexibilidad Absoluta: Diseño ‘Schemaless’ Polimórfico

En bases de datos relacionales, el esquema se define antes del software. En MongoDB, los desarrolladores insertan documentos directamente; la estructura subyacente se crea automáticamente.

```
{
   nombre: "Luis",
   cargo: "Gerente"
}
```

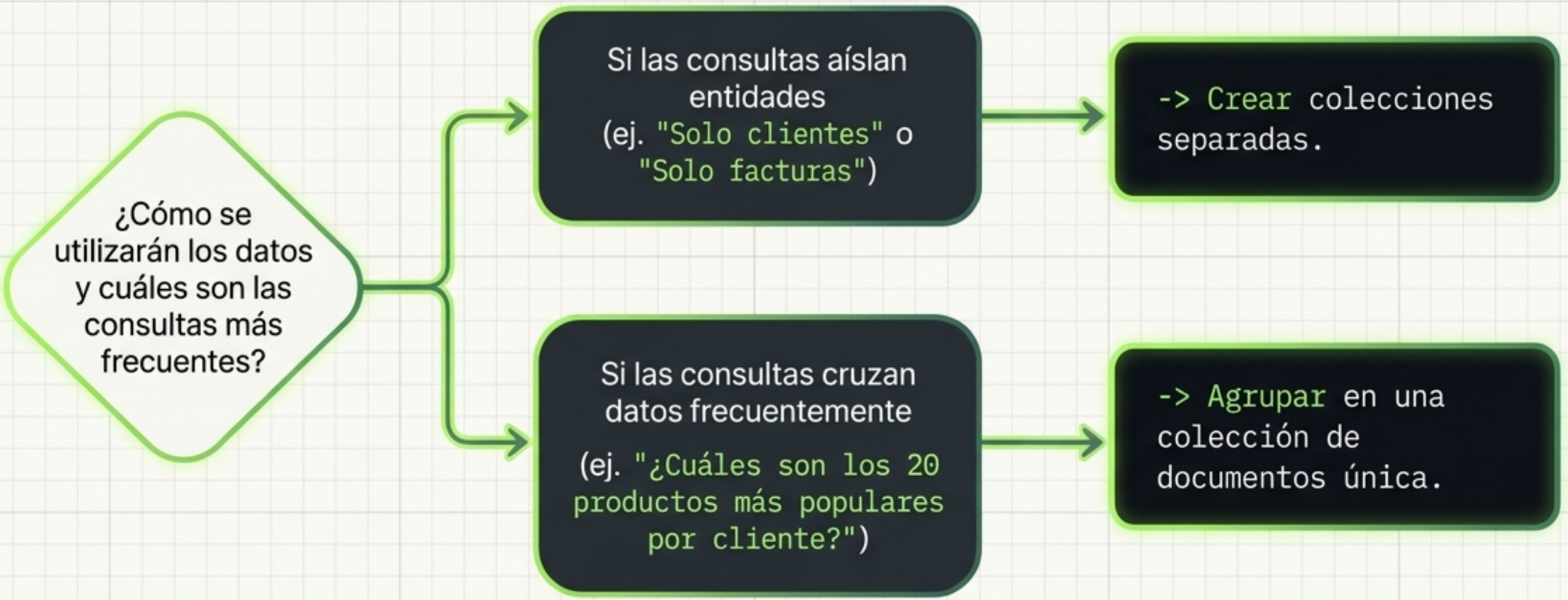
```
{
  nombre: "Roberto",
  cargo: "Asesor",
  f_alta: "1-Feb-2010",
  f_baja: "12-Ago-2014"
}
```



¡Los documentos son polimórficos! No es obligatorio que compartan la misma estructura de datos, permitiendo evolución continua sin bloqueos de base de datos.

Gemini Notebook

Arquitectura de Colecciones: El Diseño Guiado por Consultas



Los documentos en una colección no necesitan estructuras idénticas, pero deben compartir un propósito o contexto de acceso común.

Gemini Notebook

Modelado de Relaciones: Embeber vs. Referenciar



Documentos Embebidos (*Embedding*)

Cuándo usar: Relaciones **1:1** o **1:N** (donde 'N' siempre se consulta junto al padre). Agregaciones lógicas ('contiene').

Beneficio: ✅ Recuperación de información mucho más rápida (una sola operación de lectura).

Desventaja: ⚠️ Implica desnormalización. No se puede acceder al hijo directamente sin pasar por el padre.



Referencias

Cuándo usar: Relaciones **Muchos a Muchos** (**N:M**). Jerarquías complejas. Cuando 'N' es un tamaño gigantesco.

Beneficio: ✅ Evita duplicidad de datos masiva; permite acceso independiente a las entidades.

Desventaja: ⚠️ Requiere múltiples consultas para unificar los datos.

Gemini Notebook

El Equilibrio del Rendimiento: **Índices**

Identificar el número ideal de índices es crítico. No existe una regla única; depende del ratio lectura/escritura de tu aplicación.

Velocidad de Lectura



Velocidad de Escritura

BD con fuerte carga de LECTURAS

Suelen tener muchos índices. Ideal para análisis o búsquedas sin patrones claros.

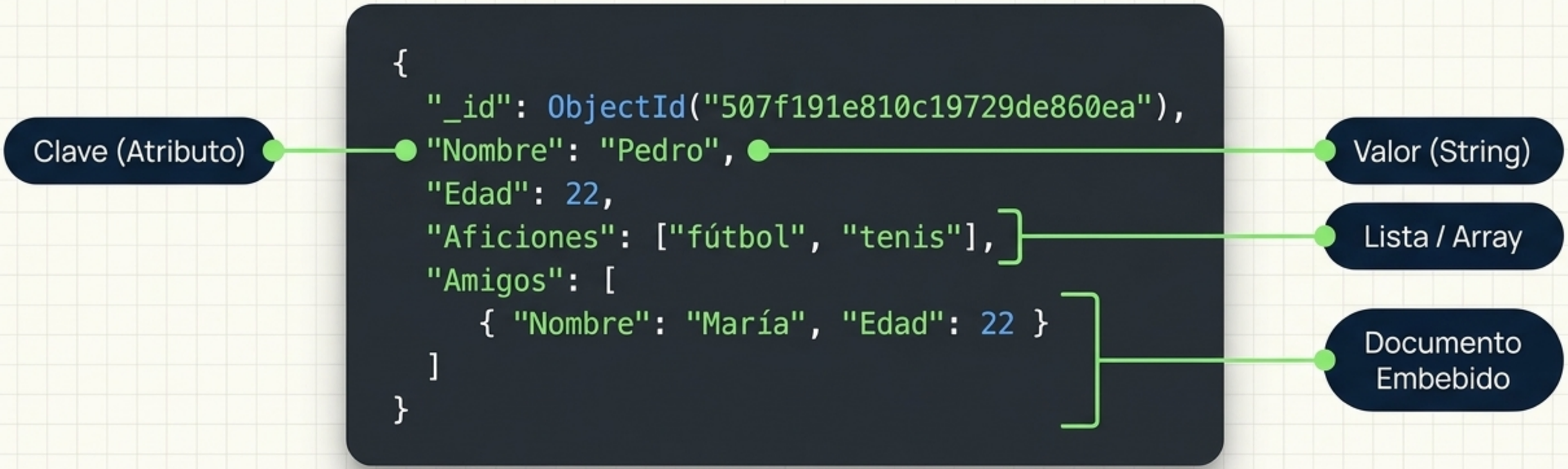
BD con fuerte carga de ESCRITURAS

Ejemplo: Sensores IoT. Requiere minimizar el número de índices para no penalizar el tiempo de inserción.

Pro-Tip: Exceso de índices = penaliza escrituras.
Falta de índices = penaliza lecturas.

Gemini Notebook

Anatomía de un Documento MongoDB (BSON)



MongoDB es la BD NoSQL más popular. Almacena documentos en BSON (Binary JSON), permitiendo estructuras anidadas que eliminan la necesidad de complejas uniones (Joins).

Gemini Notebook

Ecosistema y Herramientas de Trabajo

Component 1 (The Core)

mongod (Servidor)

El motor principal (daemon) que gestiona los datos en disco (\data\db).

Component 2 (The Native CLI)

mongo (Shell)

La consola interactiva nativa para ejecutar comandos JavaScript directamente contra el servidor.

Component 3 (The GUIs)

MongoDB Compass: Herramienta oficial para visualizar colecciones, explicar consultas gráficamente (explain plans visuales) y gestionar índices.

Robo 3T: Editor alternativo popular y ligero.

Gemini Notebook

Inicialización:
La Consola de Comandos

Arrancar el Servidor

```
> mongod.exe
// Usa el directorio por defecto C:\data\db
> mongod.exe --dbpath 'C:\datosmongo\db'
// Ruta personalizada
// Puerto por defecto: 27017
```

Conectar e Identificar

```
> mongo.exe  // Conecta por defecto a BD 'test'
> db.version()  // Verifica la versión instalada
> db    // Muestra a qué BD estás conectado actualmente
```

Navegación de Bases de Datos

```
> show databases // Lista las bases de datos
> use BDNueva   // Cambia de BD o crea una nueva al insertar el primer dato
> db.dropDatabase() // Elimina la BD actual
```

Gemini Notebook

Reglas de Diseño y Nomenclatura

Bases de Datos

- ✅ Sensible a **mayúsculas/minúsculas** (`Case sensitive`).
- ✅ Longitud máxima: **64 caracteres**.
- ✅ Recomendación: Usar sufijo "DB" o "BD".

Colecciones (Restricciones y Buenas Prácticas)

- ❌ NUNCA contener: `$` o `\0` **(nulos)**.
- ❌ NUNCA empezar con espacio o la palabra `system`.
- ✅ Preferiblemente **minúsculas** y en **plural** (ej. `clientes`, no `Cliente`).
- ✅ Evitar separadores complejos como **guiones bajos (_)** si no es estrictamente necesario.

Gemini Notebook

Operaciones CRUD: Inserción de Datos (Create)

`insertOne()`

Sintaxis:

```
db.coleccion.insertOne({
  'clave':'valor'
})
```

Uso:
Inserta un único documento.

Retorno:
Devuelve el **_id** insertado. (Desde v3.2)

`insertMany()`

Sintaxis:

```
db.coleccion.insertMany([
  {doc1}, {doc2}
])
```

Uso:
Inserta múltiples documentos pasando un **Array**.

Retorno:
Devuelve la lista de los **_id** generados. (Desde v3.2)

`insert() / save()`

Sintaxis:

```
db.coleccion.insert(
  {doc} o [Array]
)
```

Uso:
Operador clásico (**Legacy**).
Si la colección no existe, la crea automáticamente.
Asigna un **ObjectId** de 12 bytes (**_id**) si se omite.

Gemini Notebook

Comportamiento de Inserción Masiva y Límites

Límite del Sistema:
Tamaño máximo de un documento BSON = 16 MB.

Escenario 1: Inserción Ordenada (Por defecto)



Comportamiento: Inserta en secuencia. Si un documento falla (ej. _id duplicado), el proceso se detiene y los documentos restantes se descartan.

Escenario 2: Inserción No Ordenada



Comportamiento: Tolerancia a fallos. Si ocurre un error con un documento, el motor continúa con la inserción del resto del array.

Gemini Notebook

Recuperación (Read): Proyecciones con find()

Una consulta `find( {filtro}, {proyección} )` permite limitar los datos devueltos por el servidor, optimizando la red.

```
db.clientes.find({'nombre': 'Alfredo'},
  {_id: 1, ciudad: 1})
```

Inclusión (1): Solo devuelve _id y ciudad. (Nota: el _id siempre se incluye por defecto a menos que se excluya explícitamente).

```
db.clientes.find({'nombre': 'Alfredo'}, {ciudad: 0})
```

Exclusión (0): Devuelve el documento entero excepto el atributo ciudad.

```
db.clientes.find({}, {_id: 0, nombre: 1})
```

Ocultar ID: Muestra solo el nombre de todos los clientes, eliminando el ObjectId.

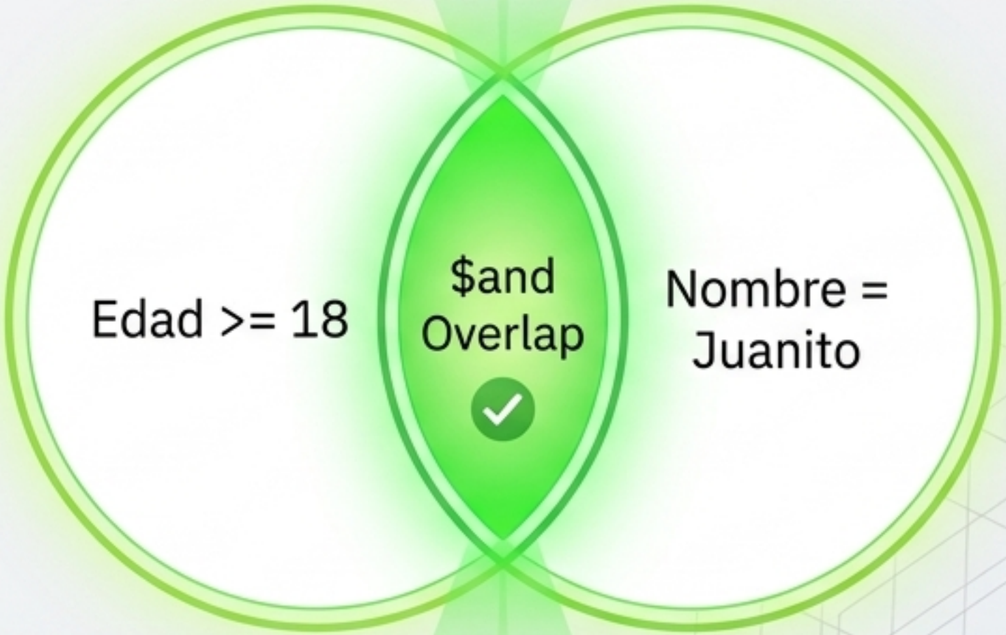


Gemini Notebook

Consultas Condicionales: Operadores Lógicos

El operador **$and** realiza una evaluación sobre un array de múltiples expresiones. Recupera únicamente los documentos que satisfacen todas las condiciones estrictamente.

```
Sintaxis Base: {
  $and: [
    { <expresión1> },
    { <expresión2> }
  ]
}
```



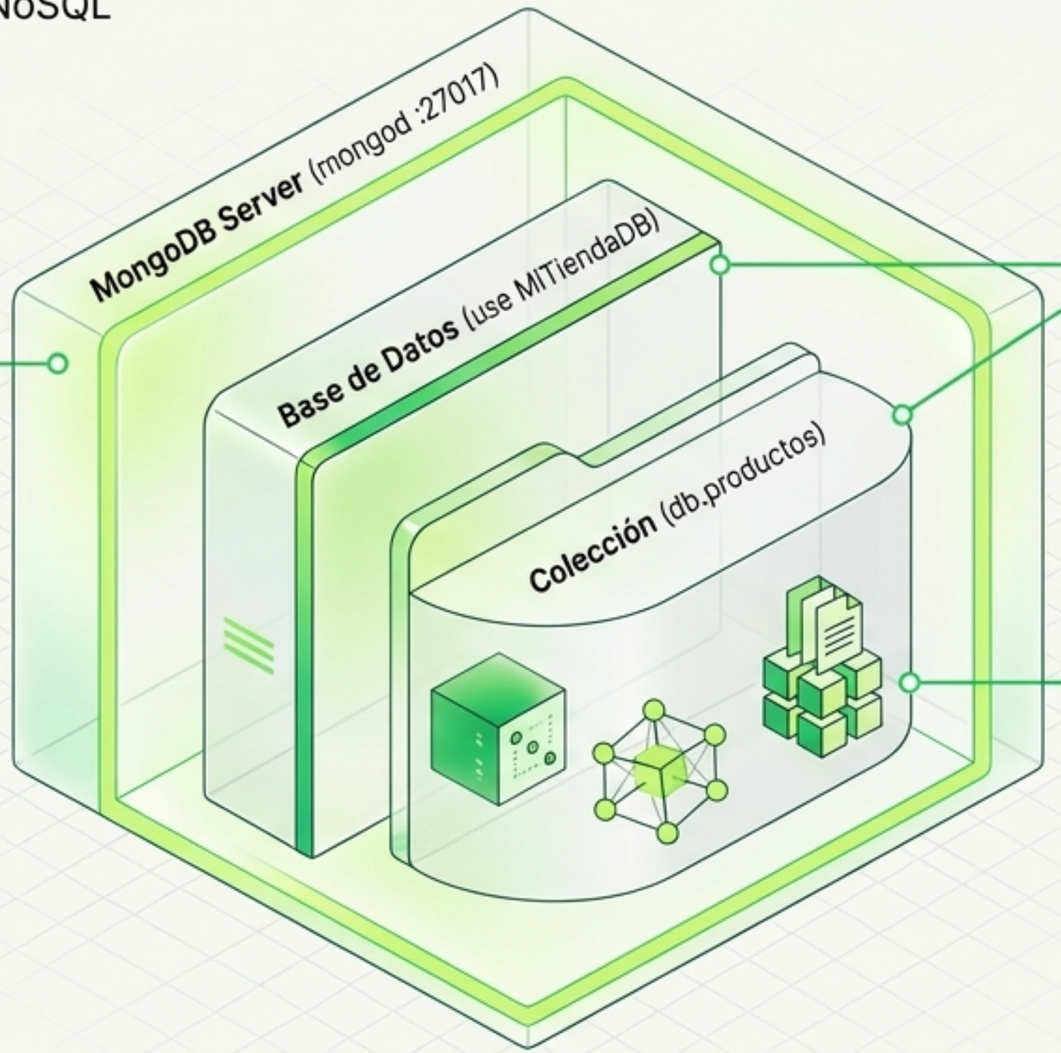

```
db.alumnos.find({
  $and: [
    { 'Edad': { $gte: 18 } },
    { 'nombre': 'Juanito' }
  ]
})
```

Traducción: "Encuentra a los alumnos que se llamen exactamente Juanito Y que tengan una edad Mayor o Igual ($gte) a 18."

Gemini Notebook

La Arquitectura Completa de Datos NoSQL

Motor de alto rendimiento y esquema dinámico (BSON).



DB & Colecciones:
Agrupación lógica sin esquemas predefinidos. Creadas dinámicamente al momento de la primera inserción (CRUD).

Documentos:
La verdadera flexibilidad. Modelado ágil usando documentos embebidos o referencias según las consultas de la aplicación.

MongoDB traslada el control del esquema de la base de datos al código de tu aplicación, dándote total libertad para construir.

Gemini Notebook